

CZECH TECHNICAL UNIVERSITY IN PRAGUE

FACULTY OF ELECTRICAL ENGINEERING
DEPARTMENT OF CYBERNETICS
MULTI-ROBOT SYSTEMS



Rigorous Evaluation of Large-scale Mapping and Localization Systems on High-fidelity Synthetic Datasets

Bachelor's Thesis

Petr Kučera

Prague, April 2025

Study programme: Cybernetics and Robotics

Supervisor: Ing. David Čapek, Ing. Tomáš Báča, Ph.D

Acknowledgments

Firstly, I would like to express my gratitude to my supervisor.

I. OSOBNÍ A STUDIJNÍ ÚDAJE

Příjmení: **Kučera** Jméno: **Petr** Osobní číslo: **524039**
Fakulta/ústav: **Fakulta elektrotechnická**
Zadávající katedra/ústav: **Katedra kybernetiky**
Studijní program: **Kybernetika a robotika**

II. ÚDAJE K BAKALÁŘSKÉ PRÁCI

Název bakalářské práce:

Rigorózní vyhodnocení systémů pro rozsáhlé mapování a lokalizaci na vysoce věrných syntetických datasetech

Název bakalářské práce anglicky:

Rigorous Evaluation of Large-scale Mapping and Localization Systems on High-fidelity Synthetic Datasets

Jméno a pracoviště vedoucí(ho) bakalářské práce:

Ing. David Čapek Multirobotické systémy FEL

Jméno a pracoviště druhé(ho) vedoucí(ho) nebo konzultanta(ky) bakalářské práce:

Ing. Tomáš Báča, Ph.D. Multirobotické systémy FEL

Datum zadání bakalářské práce: **05.02.2026**

Termín odevzdání bakalářské práce: _____

Platnost zadání bakalářské práce: **19.09.2027**



podpis vedoucí(ho) ústavu/katedry



podpis proděkana(ky) z pověření děkana(ky)

III. PŘEVZETÍ ZADÁNÍ

Student bere na vědomí, že je povinen vypracovat bakalářskou práci samostatně, bez cizí pomoci, s výjimkou poskytnutých konzultací. Seznam použité literatury, jiných pramenů a jmen konzultantů je třeba uvést v bakalářské práci.

Datum převzetí zadání

Kučera Petr

Podpis studenta

I. OSOBNÍ A STUDIJNÍ ÚDAJE

Příjmení: **Kučera** Jméno: **Petr** Osobní číslo: **524039**
Fakulta/ústav: **Fakulta elektrotechnická**
Zadávající katedra/ústav: **Katedra kybernetiky**
Studijní program: **Kybernetika a robotika**

II. ÚDAJE K BAKALÁŘSKÉ PRÁCI

Název bakalářské práce:

Rigorózní vyhodnocení systémů pro rozsáhlé mapování a lokalizaci na vysoce věrných syntetických datasetech

Název bakalářské práce anglicky:

Rigorous Evaluation of Large-scale Mapping and Localization Systems on High-fidelity Synthetic Datasets

Pokyny pro vypracování:

Autonomous robotic systems have seen significant development in recent years. Showing a shift from fine-tuned systems that were domain-specific to general systems. This led to a need for large amounts of training and testing data. The easiest way to obtain labeled data is to use paternalistic simulators. Creating novel environments for the simulators can be time-consuming. That is what this thesis will focus on: creating an automated method for generating high-fidelity environments for the verification of existing mapping and localization systems.

The thesis will be divided into the following parts:

- Familiarize yourself with the FlightForge simulator [1] and Robot Operating System 2 (ROS2).
- Research methods for creating photorealistic environments [2, 3].
- Implement or adapt the selected method into the FlightForge simulator.
- Generate datasets containing LiDARs, RGB cameras, Ground truth segmentations for both modalities using the created environments that will be used for the verification of existing mapping and localization methods [4].
- Verify the existing localization and mapping frameworks on the generated datasets.
- Prepare the dataset for public release.

Seznam doporučené literatury:

- [1]. D. Čapek, et al. "FlightForge: Advancing UAV Research with Procedural Generation of High-Fidelity Simulation and Integrated Autonomy", 2025 IEEE International Conference on Robotics & Automation (ICRA)
- [2]. Fan-Yun Sun, et al. "3D-Generalist: Self-Improving Vision-Language-Action Models for Crafting 3D Worlds", 2026 International Conference on 3D Vision
- [3]. Yan Zhuang, et al. "SimWorld-Robotics: Synthesizing Photorealistic and Dynamic Urban Environments for Multimodal Robot Navigation and Collaboration", NeurIPS 2025
- [4]. K. Ebadi et al., "Present and Future of SLAM in Extreme Environments: The DARPA SubT Challenge," in IEEE Transactions on Robotics

Declaration

I declare that presented work was developed independently, and that I have listed all sources of information used within, in accordance with the Methodical instructions for observing ethical principles in preparation of university theses.

Date

Abstract

The study of autonomous Unmanned Aerial Vehicles (UAVs) has become a prominent sub-field of mobile robotics.

Keywords Unmanned Aerial Vehicles, Automatic Control

Abstrakt

Výzkum na poli autonomních bezpilotních prostředků (UAV) se stal významným oborem mobilní robotiky.

Klíčová slova Bepilotní Prostředky, Automatické Řízení

Abbreviations

API Application Programming Interface

UAV Unmanned Aerial Vehicle

EDT Euclidean Distance Transform

SDF Signed Distance Field

PCG Procedural Content Generation Framework

API Application Programming Interface

Contents

1	Introduction	1
1.1	Related works	1
1.2	Contributions	1
1.3	Mathematical notation	1
2	Methodology	2
2.1	Binary Mask Processing	2
2.1.1	Binary Mask Creation	2
2.1.2	Euclidean Distance Transform	2
2.1.3	Signed Distance Field	3
2.1.4	Binary Mask Smoothing	4
2.2	Topology Extraction from Binary Masks	6
2.2.1	Binary Mask Skeletonization	6
2.2.2	Contour Extraction of Area Features	7
2.3	Polyline Creation from Topological Pixels	8
2.3.1	Pixel Classification	8
2.3.2	Polyline Tracing Algorithm	8
2.4	Polyline Post-processing	10
2.4.1	Polyline Stitching Algorithm	10
2.4.2	Polyline Simplification Using Douglas–Peucker Algorithm	11
2.5	Spline Construction from Polylines	12
2.5.1	Hermite and Bézier Curve Representations	12
2.5.2	Catmull–Rom Spline Construction	13
2.5.3	Curve Subdivision Using de Casteljau’s Algorithm	15
3	Plugin Design and Implementation	18
3.1	Processing Pipeline	18
3.2	Landscape Creation	19
3.3	Segmentation Processing	19
3.4	Unreal Engine Spline Creation Pages	22
3.4.1	Landscape Spline Creation	22
3.4.2	River and Lake Spline Creation	23
3.4.3	PCG Biome Core Creation	24
3.4.4	Biome Spline Creation	24
3.5	Generated Worlds	27
4	Conclusion	31
5	References	32

Chapter 1

Introduction

First, introduce the reader to the research topic. Start with the most general view and slowly converge to the particular field, sub-field, and the challenges you face. You can cite others' work here **baca2021mrs**.

1.1 Related works

This section should contain related state-of-the-art works and their relation to the author's work. We usually cite the original works like this **benallegue2008high**. You can also cite multiple papers at once like this **baca2016embedded**, **baca2021mrs**.

1.2 Contributions

This section should describe the author's contributions to the field of research.

1.3 Mathematical notation

It is a good practice to define basic mathematical notation in the introduction. See Table 1.1 for an example.

$\mathbf{x}, \boldsymbol{\alpha}$	vector, pseudo-vector, or tuple
$\hat{\mathbf{x}}, \hat{\boldsymbol{\omega}}$	unit vector or unit pseudo-vector
$\hat{\mathbf{e}}_1, \hat{\mathbf{e}}_2, \hat{\mathbf{e}}_3$	elements of the <i>standard basis</i>
$\mathbf{X}, \boldsymbol{\Omega}$	matrix
$\mathbf{I}$	identity matrix
$x = \mathbf{a}^\top \mathbf{b}$	inner product of $\mathbf{a}, \mathbf{b} \in \mathbb{R}^3$
$\mathbf{x} = \mathbf{a} \times \mathbf{b}$	cross product of $\mathbf{a}, \mathbf{b} \in \mathbb{R}^3$
$\mathbf{x} = \mathbf{a} \circ \mathbf{b}$	element-wise product of $\mathbf{a}, \mathbf{b} \in \mathbb{R}^3$
$\mathbf{x}_{(n)} = \mathbf{x}^\top \hat{\mathbf{e}}_n$	n^{th} vector element (row), $\mathbf{x}, \mathbf{e} \in \mathbb{R}^3$
$\mathbf{X}_{(a,b)}$	matrix element, (row, column)
x_d	x_d is <i>desired</i> , a reference
$\dot{x}, \ddot{x}, \dot{\ddot{x}}, \ddot{\ddot{x}}$	1 st , 2 nd , 3 rd , and 4 th time derivative of x
$x_{[n]}$	x at the sample n
$\mathbf{A}, \mathbf{B}, \mathbf{x}$	LTI system matrix, input matrix and input vector
$SO(3)$	3D special orthogonal group of rotations
$SE(3)$	$SO(3) \times \mathbb{R}^3$, special Euclidean group

Table 1.1: Mathematical notation, nomenclature and notable symbols.

Chapter 2

Methodology

2.1 Binary Mask Processing

2.1.1 Binary Mask Creation

A segmentation image is converted into a binary mask by selecting all pixels whose color lies within a specified range. This range can be adjusted by the user to obtain finer control over binary mask creation.

The result is a binary image in which white pixels, referred to as foreground, are assigned the value true, while black pixels, referred to as background, are assigned the value false. We refer to clusters of white or black pixels as regions.

2.1.2 Euclidean Distance Transform

Euclidean Distance Transform (EDT) assigns to each pixel the Euclidean distance to the nearest pixel of a specified set. This can be used for extracting widths from binary masks.

In our implementation, the cost function f is chosen so that distances are computed from foreground pixels to the nearest background pixel. This is determined by which pixels are assigned the values 0 and ∞ in the cost function.

This process can be generalized to transform a cost function on a grid using spatial information [3], such as using distance function other than Euclidean, or n -dimensional grids.

We define cost function as $f(q)$:

$$f(q) = \begin{cases} \infty & \text{if } q \text{ is a foreground pixel,} \\ 0 & \text{otherwise,} \end{cases} \quad (2.1)$$

where q is a pixel of the binary mask. The distance transform of f is denoted by $\mathcal{D}_f$ and defined as

$$\mathcal{D}_f(p) = \min_{q \in \mathcal{G}} (d(p, q) + f(q)) \quad , \quad (2.2)$$

where $\mathcal{G}$ is the grid representing the binary mask, $p \in \mathcal{G}$, and $d(p, q)$ is a distance function. If $d(p, q)$ is chosen as Euclidean distance, then $\mathcal{D}_f$ is called the Euclidean distance transform.

Applying EDT to the binary mask yields an unsigned distance field, shown on figure 2.1. The value at each element of the distance field is the distance from the corresponding pixel in the binary mask to the nearest background pixel. This can be seen in definition of function $f(q)$, where its value for black pixels is zero, meaning $\mathcal{G}$ computes distance of a white pixel to a black pixel. Consequently, the value of every background pixel in this field is 0.

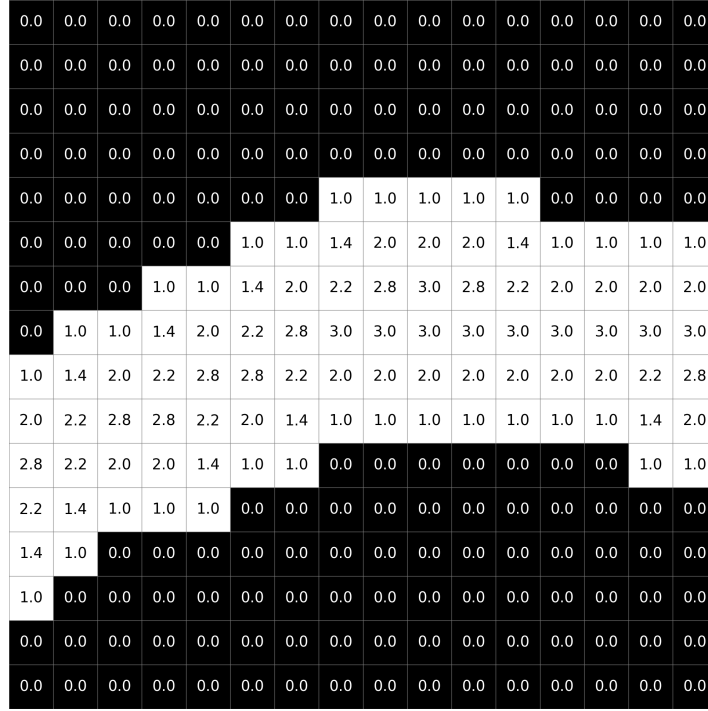


Figure 2.1: Visualization of unsigned distance field of a binary mask.

In our application, this algorithm is used to extract the widths of roads or rivers by computing EDT of foreground pixels. These widths can then be converted to approximate real-world measurements. The precision of this approach depends on the accuracy and resolution of the binary mask.

2.1.3 Signed Distance Field

Signed Distance Field (SDF), is a field in which the value corresponding to each pixel of binary mask represents the distance from that pixel to the nearest boundary between foreground and background region. Unlike the unsigned distance field, the SDF stores meaningful values on both sides of the foreground boundary. To differentiate between the two colors, we denote the distances of background pixels as negative.

First we define distance transform $\mathcal{D}_g$ for background pixels by inverting the cost function, so we compute distances of black pixel to a white pixel. The cost function $g(q)$ is

$$g(q) = \begin{cases} \infty & \text{if } q \text{ is a background pixel,} \\ 0 & \text{otherwise,} \end{cases} \quad (2.3)$$

where q is a pixel of the binary mask. The distance transform of g is denoted by $\mathcal{D}_g$ and defined as

$$\mathcal{D}_g(p) = -\min_{q \in \mathcal{G}} (d(p, q) + g(q)) \quad , \quad (2.4)$$

where $\mathcal{G}$ is the grid representing the binary mask, $p \in \mathcal{G}$, and $d(p, q)$ is the Euclidean distance.

Signed distance field, shown on figure 2.2, has the same dimensions as the initial binary mask. The SDF value v at pixel p is defined as the sum of these two distance transforms

$$v = \mathcal{D}_f(p) + \mathcal{D}_g(p), \quad (2.5)$$

where p is a pixel on binary mask. When p is a foreground pixel, $\mathcal{D}_f(p)$ is positive, while $\mathcal{D}_g(p) = 0$. Conversely, when p is a background pixel, $\mathcal{D}_f(p) = 0$ and $\mathcal{D}_g(p)$ is a negative value.

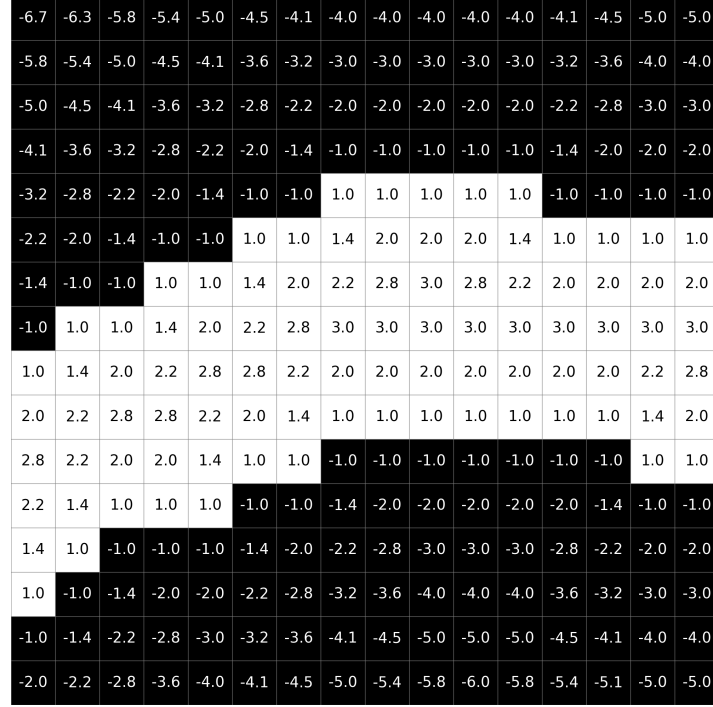


Figure 2.2: Visualization of signed distance field of a binary mask.

2.1.4 Binary Mask Smoothing

Binary mask smoothing is achieved by applying Gaussian blur to the computed signed distance field. The blurred field is then classified by a threshold. Values above a this threshold are classified as foreground, while the remaining values are classified as background. When applied to very thin foreground regions, this process may disrupt their continuity.

Gaussian function for two dimensions is defined as

$$G(x, y) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{x^2 + y^2}{2\sigma^2}\right) \quad , \quad (2.6)$$

where parameter σ determines the blur strength. Increasing σ results in stronger smoothing, while decreasing produces weaker smoothing.

To blur an image $I(x, y)$ we compute its convolution with Gaussian function

$$I_{\text{blurred}}(x, y) = (I * G)(x, y) = \sum_u \sum_v I(x - u, y - v) G(u, v). \quad (2.7)$$

In discrete implementation we use kernel of size $(2R + 1) \times (2R + 1)$, where R is a radius generally chosen as

$$R = \max(1, \lceil 3\sigma \rceil) \quad , \quad (2.8)$$

because the value of Gaussian function outside the interval $[-3\sigma, +3\sigma]$ is relatively small.

If we use computed SDF of binary mask as our image, then we get resulting blurred SDF as

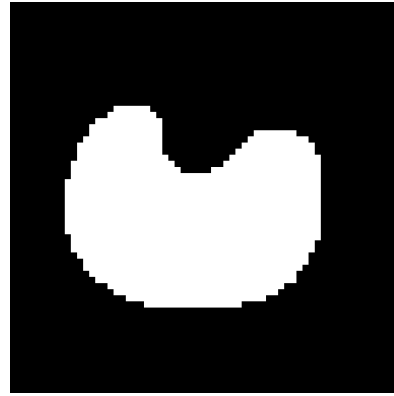
$$\text{SDF}_{\text{blurred}}(x, y) = \sum_{u=-R}^R \sum_{v=-R}^R \text{SDF}(x - u, y - v) G(u, v) \quad , \quad (2.9)$$

where $\text{SDF}(x, y)$ is an element of SDF at x and y coordinate, and $0 \leq x < \text{width}$ and $0 \leq y < \text{height}$.

The user is given option to adjust both Gaussian blur parameter σ the threshold value. Increasing the threshold erodes the resulting mask, shown on figure 2.3c, whereas decreasing it expands the foreground region, shown on figure 2.3d.



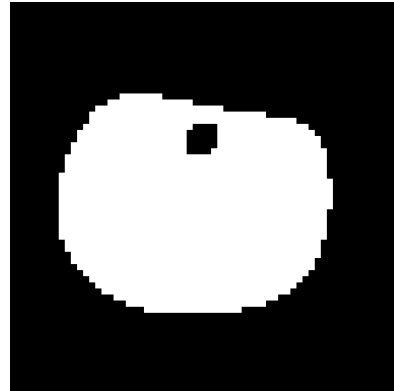
(a) Binary mask input.



(b) Binary mask smoothing with $\sigma = 5.0$.



(c) Eroded binary mask with threshold = 1.5.



(d) Expanded binary mask with threshold = -1.0.

Figure 2.3: Examples of smoothed binary masks. Input binary mask (a) is smoothed to (b) with Gaussian parameter $\sigma = 5.0$. Smoothed binary mask (b) is eroded (c) with threshold = 1.5 and is expanded (d) with threshold = -1.0.

2.2 Topology Extraction from Binary Masks

2.2.1 Binary Mask Skeletonization

Thinning of a binary mask, referred to as skeletonization, is a process of removing boundary pixels of a foreground region while maintaining their basic structure, general shape and connectivity. For algorithms used to create binary mask skeleton used for this application, we need to ensure satisfaction of certain criteria:

- The resulting line should be 1 pixel wide with no redundant pixels
- Connectivity of individual components must be preserved
- The skeleton should be approximately centered with respect to the local thickness of the component.
- There should not be substantial gap between skeleton and edge of binary mask, if the foreground region is touching the side.

Many thinning algorithms have been proposed for various applications, one of the popular ones are Guo–Hall algorithm [4] and Zhang–Suen algorithm [10]. Their main advantage is their ability to be parallelized, but both of them do not guarantee desired 1-pixel width without some additional post-processing. A different algorithm was selected [2] made by Jain and Kumar, which is sequential and guarantees skeleton with no redundant pixels and 1-pixel width. Difference between each of those algorithms can be seen on Fig. 2.4.

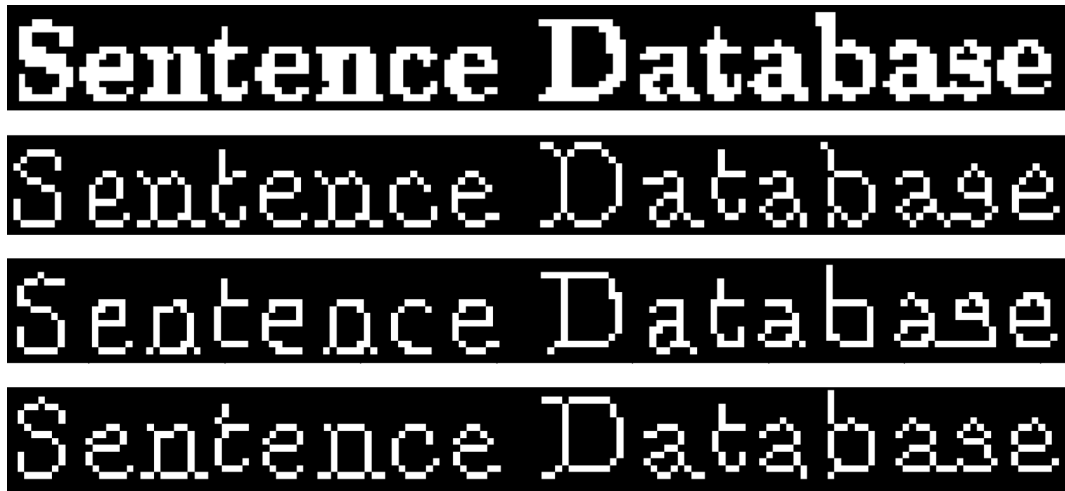
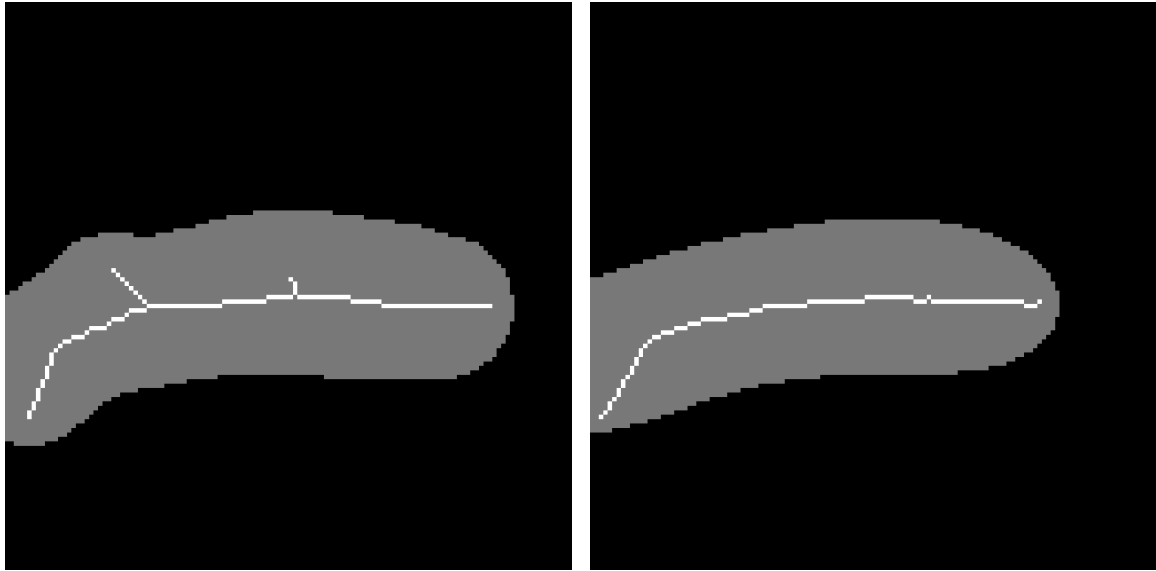


Figure 2.4: Comparison between different skeletonization algorithms. Image was taken from [2]. From top to bottom: (a) Original pattern, (b) Guo–Hall algorithm, (c) Zhang–Suen algorithm, (d) Jain–Kumar algorithm.

This algorithm works by performing passes in all four directions and sequentially removes redundant pixels until only the skeleton remains. This results in the skeleton being offset by 1 pixel from center, but this error is insignificant and does not negatively affect the final result.

Disadvantage of skeletonization algorithms is that they are sensitive to binary mask "corners". As the algorithm creates skeleton to be in the middle, when some section of the foreground region is sharper, it can create branch towards this corner. This can be resolved by smoothing the binary mask before performing skeletonization. Despite the mask slightly changing shape due to the smoothing, the general shape of the skeleton is preserved.

As seen on 2.5b, even performing skeletonization on smoothed binary mask can lead to undesirable curving on the skeleton. This level of control can not be achieved by simple binary mask smoothing but other more complex algorithms would be needed. However, this results is acceptable for our application of creating splines, as the user has the option to manually adjust them, if the error is too noticeable.



(a) Skeletonization performed on binary mask

(b) Skeletonization performed on smoothed binary mask

Figure 2.5: Example of resulting binary mask skeleton without (a) smoothing and with smoothing (b). The gray area is the binary mask, either initial or smoothed. Smoothing was performed with $\sigma = 15.0$ and threshold -1.2 .

2.2.2 Contour Extraction of Area Features

Extracting contours of binary mask foreground region is used for creating features that fill area, such as biomes or lakes.

Used algorithm cycles through every pixel of binary mask. If the pixel is black, that means in background, we skip it. If the pixel is white, in foreground, we count how many 4-neighbors are black and if there are any, we mark this pixel as contour. Pixels that are 4-neighbors are the pixels directly above, below, left or right of the initial pixel.

This algorithm skips foreground pixels along the edges of binary mask, as there are no background pixels. To create edge contour we count pixels that are outside the binary mask bounds as background.

This algorithm, although simple, reliably detects contours. There are a few scenarios where strictly 1-pixel width of the contours is not ensured. These scenarios are hard to resolve by algorithm as their outcome can rapidly change the shape or size of the contours. We decided to leave the resolution of these situations to user by giving them tools for manual edit.

2.3 Polyline Creation from Topological Pixels

A polyline is represented as an ordered list of points $\mathbf{P}_0, \mathbf{P}_1, \dots, \mathbf{P}_n$ where each consecutive pair of points $(\mathbf{P}_i, \mathbf{P}_{i+1})$, where $i = 0, 1, \dots, n$, defines one line segment. In this way, the polyline forms a connected piecewise curve composed of the segments

$$(\mathbf{P}_0, \mathbf{P}_1), (\mathbf{P}_1, \mathbf{P}_2), \dots, (\mathbf{P}_{n-1}, \mathbf{P}_n).$$

Since the points are ordered, the connectivity of the polyline is given implicitly by their order in the list.

Special case of polyline is a closed polyline. A polyline is considered closed if its first and last point are identical,

$$\mathbf{P}_0 = \mathbf{P}_n. \quad (2.10)$$

2.3.1 Pixel Classification

To create polylines we need to classify topological pixels, determining whether a pixel is an endpoint or a junction. We use a simple classification method based on counting the number of foreground pixels in the 8-neighborhood of the examined pixel.

If the number of foreground neighbors is one, the pixel is classified as an endpoint, since it is connected to only one neighboring pixel. If the number is two, the pixel is considered a regular polyline pixel connecting two neighboring pixels.

A junction pixel has three or more foreground neighbors, although in typical cases it has exactly three. If four or more foreground neighbors are present, this may indicate a more complex local structure, such as a plateau or another shape that can complicate polyline extraction.

2.3.2 Polyline Tracing Algorithm

The extracted topology, whether obtained from a binary mask skeleton or from a contour, is still represented as an unordered set of foreground pixels in the image grid. For subsequent processing, such as polyline simplification or spline construction, this discrete representation must be converted into one or more ordered polylines.

The goal of the tracing algorithm is therefore to transform the foreground topology pixels into a set of ordered polylines that collectively cover the entire topology. To achieve this, the algorithm uses the previously detected endpoint and junction pixels together with a visited map. The visited map records whether a foreground pixel has already been assigned to a traced polyline, which prevents repeated processing of the same branch. Junction pixels require special handling, since a single junction may belong to multiple output polylines.

The tracing is divided into three steps. First, all branches starting at endpoints are traced until another endpoint or a junction is reached. Second, the remaining branches connecting pairs of junctions are traced. Finally, any still unvisited foreground pixels are assumed to belong to closed contours and are processed separately. This ordering is important, because endpoint-based branches are the least ambiguous, while closed contours can only be identified reliably after all open branches have already been removed.

When tracing from a current pixel, the algorithm examines its 8-neighborhood and selects the next valid foreground pixel that has not yet been visited. If multiple candidates

exist, endpoint or junction pixels are preferred over regular foreground pixels. Among such candidates, the closest one is selected according to Manhattan distance. This rule improves local continuity of the traced polyline and reduces short redundant detours that may otherwise arise in the discrete pixel representation.

Algorithm 1: Topology tracing into ordered polylines

```

1: procedure TRACE_TOPOLOGY(foreground_pixels, endpoints, junctions)
2:   visited  $\leftarrow$  map initialized to false
3:   result  $\leftarrow$  empty list of polylines
4:
5:   for all endpoint from endpoints do
6:     if not visited[ endpoint ] then
7:       trace a polyline starting at endpoint
8:       stop when another endpoint or a junction is reached
9:       mark regular traced pixels as visited
10:      add the resulting polyline to result
11:    end if
12:  end for
13:
14:  for all junction from junctions do
15:    for all unvisited foreground neighbors of junction do
16:      trace a polyline starting at junction
17:      stop when another junction is reached
18:      mark regular traced pixels as visited
19:      add the resulting polyline to result
20:    end for
21:  end for
22:
23:  for all unvisited foreground pixels p do
24:    trace a closed contour starting at p
25:    continue until no unvisited foreground neighbor exists
26:    append the starting point again to represent closure
27:    add the resulting polyline to result
28:  end for
29:  return result
30: end procedure

```

Tracing from endpoints. In the first stage, each unvisited endpoint is used as a starting point of a new polyline. The tracing then proceeds through unvisited foreground pixels until either another endpoint or a junction is reached. Regular foreground pixels are appended to the polyline and marked as visited. If the traced branch terminates at a junction, the junction is appended to the polyline but is not marked as visited, since it may also serve as a connection point for other branches.

Tracing between junctions. After all endpoint-based branches have been processed, some foreground pixels may still remain unvisited. In a skeleton topology, these pixels are typically expected to form branches connecting one junction to another. Since a single junction may have multiple outgoing branches, each of its unvisited foreground neighbors is considered

separately. For each such neighbor, a new polyline is initialized with the starting junction as its first point and traced until another junction is reached. As in the previous case, only regular foreground pixels are marked as visited during the traversal.

Tracing closed contours. After the previous two stages, any remaining unvisited foreground pixels are assumed to belong to closed contours with no endpoints and no junctions. This case occurs most commonly when tracing contours of foreground regions rather than binary mask skeletons. The tracing starts from any remaining unvisited foreground pixel and continues through unvisited foreground neighbors until the contour has been fully traversed. To represent closure explicitly, the starting point is appended once more at the end of the resulting polyline so that the first and last point are identical.

2.4 Polyline Post-processing

2.4.1 Polyline Stitching Algorithm

Polyline stitching is the process of connecting two or more polylines by merging pairs of endpoints, resulting in one or more longer continuous polylines. A distance threshold can be specified that determines when two endpoints are considered close enough to be merged.

The proposed algorithm follows a greedy strategy. In each iteration, it considers all endpoints of currently active polylines and finds the closest valid pair whose distance does not exceed the specified threshold. The corresponding polylines are then merged into a new polyline, while one of the original polylines is marked as inactive. This process is repeated until no further valid merge can be found.

Algorithm 2: Polyline Stitching algorithm

```

1: procedure STITCH_POLYLINES(polylines, max_distance)
2:   resulting_polylines  $\leftarrow$  initialize empty list
3:   used_polylines  $\leftarrow$  initialize list with false values, length of polylines
4:   polyline_ends  $\leftarrow$  initialize empty list of points
5:
6:   while true do
7:     polyline_ends.reset()
8:
9:     for polyline from polylines do
10:      if not used_polylines[ polyline ] then
11:        polyline_ends.add(polyline.first)
12:        polyline_ends.add(polyline.last)
13:      end if
14:    end for
15:
16:    best_pair  $\leftarrow$  initialize
17:    best_distance =  $\infty$ 
18:    for first_end from polyline_ends do
19:      for second_end from polyline_ends do
20:        if if both ends are not from the same polyline then
21:          distance =  $||\textit{first\_end} - \textit{second\_end}||$ 

```

```

22:
23:         if  $distance \leq max\_distance$  and  $distance < best\_distance$  then
24:              $best\_pair = (first\_end, second\_end)$ 
25:              $best\_distance = distance$ 
26:         end if
27:     end if
28: end for
29: end for
30: if  $best\_pair$  not chosen then
31:     break
32: end if
33:
34:     construct  $merged\_polyline$  from the  $best\_pair$ 
35:     reverse one polyline if necessary
36:     remove the duplicate connecting endpoint
37:
38:      $polylines[index\ of\ best\_pair.first] = merged\_polyline$ 
39:      $used\_polylines[best\_pair.second] = true$ 
40: end while
41:
42: for  $polyline$  from  $polylines$  do
43:     if not  $used\_polylines[polyline]$  then
44:          $resulting\_polylines.add(polyline)$ 
45:     end if
46: end for
47: return  $resulting\_polylines$ 
48: end procedure

```

Depending on which endpoints are connected, one of the polylines may need to be reversed before merging in order to preserve a consistent point ordering. When polylines are merged, one of the connecting endpoints must be removed, as it would result in a duplicate point and other algorithms may not work as intended.

In the implementation, an additional iteration limit can be used as a safety measure to prevent unexpected infinite execution in degenerate cases.

2.4.2 Polyline Simplification Using Douglas–Peucker Algorithm

Polyline created from binary mask skeleton or contour can have large amount of redundant points, which could hinder subsequent spline interpolation. To solve this, we employ Douglas–Peucker algorithm [6] for simplifying polylines.

Distance parameter $\epsilon > 0$ is chosen. The algorithm is initially given first and last point of the polyline and both of them are marked to be kept. Line segment is formed between them. It then examines all the remaining points and finds with the maximum distance to this line segment. If this distance is greater than ϵ , the point is marked to be kept and the algorithm is recursively applied to the two sub-polylines defined by the first point, the selected point, and the last point. Example of reduced polyline by Douglas–Peucker algorithm can be seen on figure 2.6.

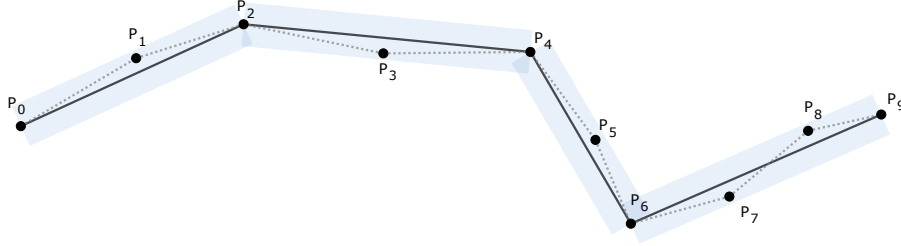


Figure 2.6: Visual example of Douglas–Peucker algorithm. Initial polyline is drawn in gray dotted line, final polyline is black. Blue rectangles are visualization of parameter ϵ .

When the recursion is completed a new polylines is constructed from the points which are marked to be kept. Resulting polyline is of the same orientation and approximates the shape of the initial polyline, depending on the parameter ϵ . This simplification step reduces the number of control points while preserving the overall shape of the extracted topology, which makes the subsequent spline construction more stable and easier to control.

2.5 Spline Construction from Polylines

2.5.1 Hermite and Bézier Curve Representations

In our implementation, we chose to represent splines as a sequence of cubic Bézier curves rather than as Hermite curves, which are used internally by Unreal Engine 5. The main reason for this choice is that Bézier curves are more convenient for the algorithms introduced later in this chapter. At the same time, a cubic Bézier curve can be converted to a Hermite representation in a straightforward manner.

A cubic Bézier curve $\mathbf{B}(t)$, shown on figure 2.7, is defined by two end points, $\mathbf{P}_0$ and $\mathbf{P}_3$, and two control points, $\mathbf{P}_1$ and $\mathbf{P}_2$. It is given by

$$\mathbf{B}(t) = (1-t)^3\mathbf{P}_0 + 3(1-t)^2t\mathbf{P}_1 + 3(1-t)t^2\mathbf{P}_2 + t^3\mathbf{P}_3 \quad , \quad (2.11)$$

for $t \in [0, 1]$.

A Hermite curve $\mathbf{H}(t)$, shown on figure 2.8, is defined by two end points, $\mathbf{H}_0$ and $\mathbf{H}_1$, and two tangent vectors, $\mathbf{M}_0$ and $\mathbf{M}_1$. It is given by

$$\mathbf{H}(t) = (2t^3 - 3t^2 + 1)\mathbf{H}_0 + (t^3 - 2t^2 + t)\mathbf{M}_0 + (-2t^3 + 3t^2)\mathbf{H}_1 + (t^3 - t^2)\mathbf{M}_1 \quad , \quad (2.12)$$

for $t \in [0, 1]$.

A Hermite curve defined by end points $\mathbf{H}_0$, $\mathbf{H}_1$ and tangent vectors $\mathbf{M}_0$, $\mathbf{M}_1$ can be constructed from a cubic Bézier curve defined by control points $\mathbf{P}_0$, $\mathbf{P}_1$, $\mathbf{P}_2$, and $\mathbf{P}_3$ using the following relationships:

$$\begin{aligned} \mathbf{H}_0 &= \mathbf{P}_0 \quad , \\ \mathbf{M}_0 &= 3(\mathbf{P}_1 - \mathbf{P}_0) \quad , \\ \mathbf{M}_1 &= 3(\mathbf{P}_3 - \mathbf{P}_2) \quad , \\ \mathbf{H}_1 &= \mathbf{P}_3. \end{aligned} \quad (2.13)$$

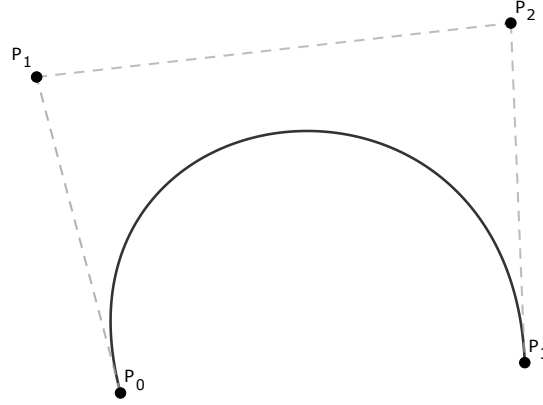


Figure 2.7: Example of cubic Bézier curve constructed from four control points.

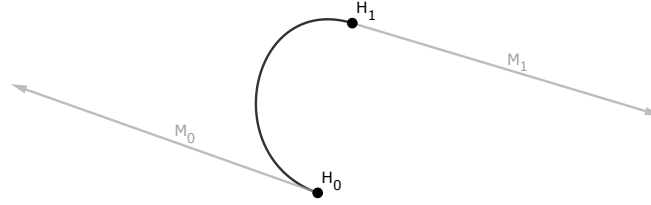


Figure 2.8: Example of Hermite spline constructed from two points and two tangent vectors.

2.5.2 Catmull–Rom Spline Construction

To interpolate spline through points of a polyline, we employed a Catmull–Rom spline. Catmull–Rom splines are a family of cubic interpolating splines constructed such that a tangent at point $\mathbf{P}_i$ is computed using the previous point $\mathbf{P}_{i-1}$ and the next point $\mathbf{P}_{i+1}$.

We used a class of parameterization described in [5] of Catmull–Rom spline. Let the knot sequence satisfy

$$t_{k+1} = t_k + \|\mathbf{P}_{k+1} - \mathbf{P}_k\|^\alpha, \quad (2.14)$$

where $\alpha \in [0, 1]$ and $t_0 = 0$. If $\alpha = 0$ then the parameterization is uniform, for $\alpha = 1$ the parameterization becomes chordal. Example of spline interpolation through polyline defined by points $\mathbf{P}_0$ to $\mathbf{P}_5$ can be seen on figure 2.9.

In our implementation, we use $\alpha = 0.5$, which corresponds to the centripetal parameterization of Catmull–Rom spline. This variant was selected because it is the only parameterization in this family that guarantees no cusps or self-intersections within a single curve segment [9].

As follows from the definition, a Catmull–Rom spline segment is constructed from four points, $\mathbf{P}_0$ to $\mathbf{P}_3$, and produces a curve segment between $\mathbf{P}_1$ and $\mathbf{P}_2$. For an open polyline, this means that the spline would not naturally interpolate the first and the last point of the polyline. To resolve this issue, we introduce auxiliary end points defined as

$$\begin{aligned} \mathbf{P}_{-1} &= 2\mathbf{P}_0 - \mathbf{P}_1, \\ \mathbf{P}_{n+1} &= 2\mathbf{P}_n - \mathbf{P}_{n-1}, \end{aligned} \quad (2.15)$$

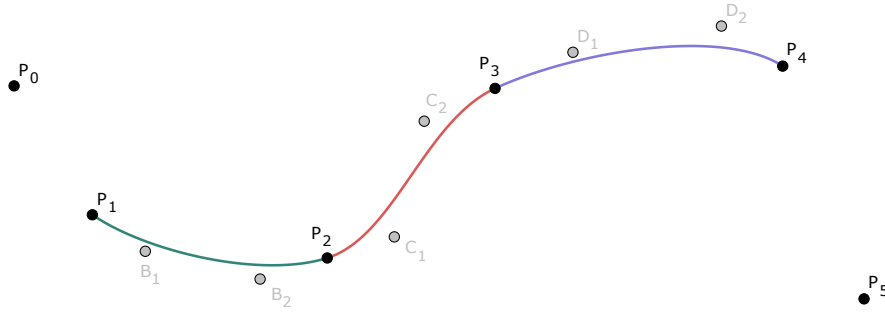


Figure 2.9: Example of spline interpolation by Catmull–Rom spline. Points $B_1, B_2, C_1, C_2, D_1, D_2$ are corresponding control points for Bézier curve.

where the polyline consists of points $P_0, \dots, P_n$. Example of such extension can be seen on figure 2.10.

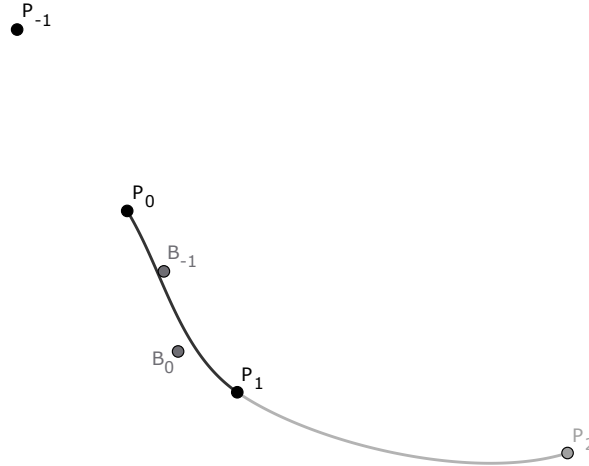


Figure 2.10: Example of Catmull–Rom endpoint extension.

A drawback of this endpoint extension is that it may produce undesirable shapes when the polyline forms a sharp turn near the boundary. As shown in Fig. 2.11, the first spline segment is constructed from the points P_{-1}, P_0, P_1 , and P_2 . Since P_{-1} is defined to be collinear with P_0 and P_1 , the tangent of the segment between P_0 and P_1 at P_0 is aligned with the line passing through these points. As a result, the curve segment is pulled in the tangent direction, which may lead to an unnatural shape near the endpoint. However, since the base polylines are constructed from binary mask skeletons or contours, this situation occurs only rarely in practice.

The knot sequence in Equation (2.14) determines the parameter spacing between interpolation points. Unlike the Hermite or Bézier formulations, it does not directly provide a

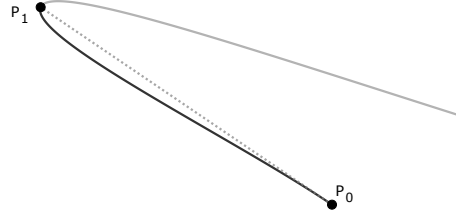


Figure 2.11: Example of Catmull–Rom endpoint extension sharp curve.

polynomial expression of the curve segment. However, this parameterization can be used to construct equivalent Hermite or Bézier representations of the same curve segment.

The Bézier curve representation of a Catmull–Rom spline segment can be computed directly from the four defining points, $\mathbf{P}_0$ to $\mathbf{P}_3$. If the Bézier curve is defined by control points $\mathbf{B}_0$ to $\mathbf{B}_3$, the relationship is given by [9]

$$\begin{aligned} \mathbf{B}_0 &= \mathbf{P}_1, \\ \mathbf{B}_1 &= \frac{d_1^{2\alpha} \mathbf{P}_2 - d_2^{2\alpha} \mathbf{P}_0 + (2d_1^{2\alpha} + 3d_1^\alpha d_2^\alpha + d_2^{2\alpha}) \mathbf{P}_1}{3d_1^\alpha (d_1^\alpha + d_2^\alpha)}, \\ \mathbf{B}_2 &= \frac{d_3^{2\alpha} \mathbf{P}_1 - d_2^{2\alpha} \mathbf{P}_3 + (2d_3^{2\alpha} + 3d_3^\alpha d_2^\alpha + d_2^{2\alpha}) \mathbf{P}_2}{3d_3^\alpha (d_3^\alpha + d_2^\alpha)}, \\ \mathbf{B}_3 &= \mathbf{P}_2, \end{aligned} \quad (2.16)$$

where d_1 , d_2 , and d_3 are defined as

$$\begin{aligned} d_1 &= \|\mathbf{P}_1 - \mathbf{P}_0\|, \\ d_2 &= \|\mathbf{P}_2 - \mathbf{P}_1\|, \\ d_3 &= \|\mathbf{P}_3 - \mathbf{P}_2\|, \end{aligned} \quad (2.17)$$

and $\|\cdot\|$ denotes the Euclidean norm. The knot intervals corresponding to the distances d_1 , d_2 , and d_3 satisfy

$$\begin{aligned} t_1 - t_0 &= d_1^\alpha, \\ t_2 - t_1 &= d_2^\alpha, \\ t_3 - t_2 &= d_3^\alpha. \end{aligned} \quad (2.18)$$

2.5.3 Curve Subdivision Using de Casteljau's Algorithm

Bézier curve can be constructed recursively by using de Casteljau's algorithm. This method is numerically more stable, than using polynomial definition (2.11). Another important use case of this algorithm is subdividing Bézier curve into two segments and subsequently getting correct their control points.

Let cubic Bézier curve $\mathbf{B}(t)$ be defined by four control points $\mathbf{P}_0$ to $\mathbf{P}_3$ and $t \in [0, 1]$. De Casteljau's algorithm for cubic Bézier curve $\mathbf{B}$ works by performing three steps. For parameter $u \in [0, 1]$ it creates points

$$\mathbf{Q}_i = (\mathbf{P}_{i+1} - \mathbf{P}_i)u + \mathbf{P}_i, \quad (2.19)$$

for $i = 0, 1, 2$. Point $\mathbf{Q}_i$ is a point on a line connecting control points $\mathbf{P}_i$ and $\mathbf{P}_{i+1}$. Then it creates another points

$$\mathbf{R}_j = (\mathbf{Q}_{j+1} - \mathbf{Q}_j)u + \mathbf{Q}_j, \quad (2.20)$$

for $j = 0, 1$. Similarly as with $\mathbf{Q}_i$, point $\mathbf{R}_j$ is a point on a line connecting points $\mathbf{Q}_j$ and $\mathbf{Q}_{j+1}$. Final point is constructed on a line between $\mathbf{R}_0$ and $\mathbf{R}_1$ as

$$\mathbf{S} = (\mathbf{R}_1 - \mathbf{R}_0)u + \mathbf{R}_0. \quad (2.21)$$

Final point $\mathbf{S}$ lies on a cubic Bézier $\mathbf{B}(t)$ and is equal to a point $\mathbf{B}(u)$. Algorithm is visualized on figure 2.12

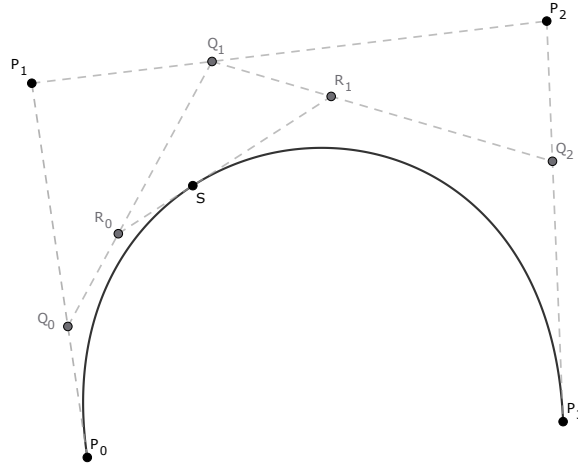


Figure 2.12: Visual example of de Casteljau algorithm for $u = 0.35$.

We can then create two cubic Bézier curves $\mathbf{B}_A(t_A)$ and $\mathbf{B}_B(t_B)$, and $t_A, t_B \in [0, 1]$. Bézier curve $\mathbf{B}_A$, visualized in figure 2.13a, is defined by control points $\mathbf{P}_0, \mathbf{Q}_0, \mathbf{R}_0$ and $\mathbf{S}$. Bézier curve $\mathbf{B}_B$, visualized in figure 2.13b, is defined by control points $\mathbf{S}, \mathbf{R}_1, \mathbf{Q}_2$ and $\mathbf{P}_3$. These segments when combined results in the initial Bézier curve $\mathbf{B}(t)$.

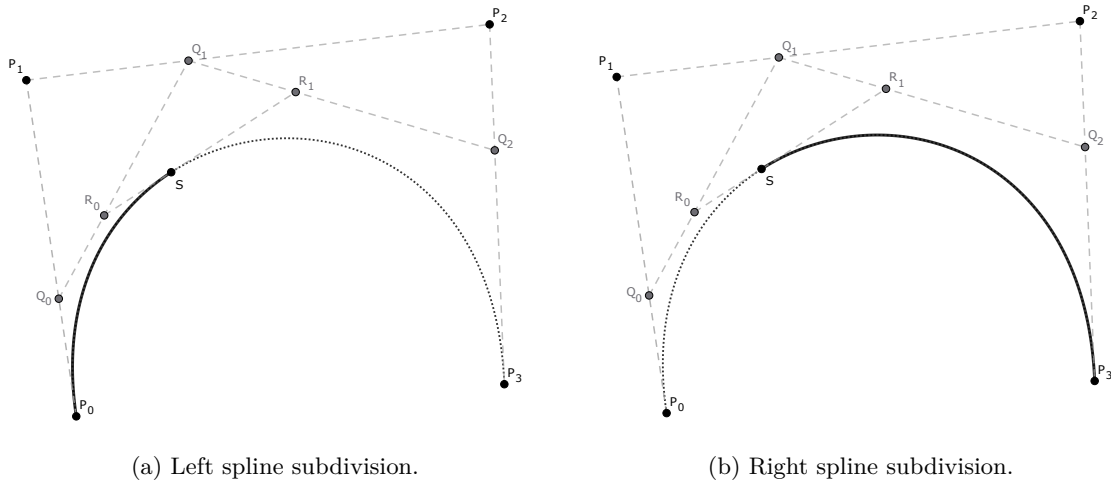


Figure 2.13: Example of subdividing Bézier curve using de Casteljau algorithm for $u = 0.35$, the left subdivision (a) and the right subdivision (b).

Chapter 3

Plugin Design and Implementation

In the previous chapter, the algorithms and methodology used in the proposed Unreal Engine 5 plugin were described. The plugin depends on several official Unreal Engine plugins, namely Water, Procedural Content Generation Framework (PCG) Framework [8], and PCG Biome Core [7]. The latter two provide built-in support for procedural generation, while the Water plugin enables correct landscape deformation for rivers and lakes. Plugin was build in Unreal Engine 5 version 5.4.4.

This plugin is used to create simulation environments for the Flight Forge Simulator [1]. It was designed as an editor plugin, which means that it relies on tools and functionality that are not available at Unreal Engine runtime. As a result, the generated environments must either be compiled into the simulator or loaded from external data prepared in advance.

3.1 Processing Pipeline

The intention of the processing pipeline was its simplicity. User can easily create landscape splines, rivers, lakes and biomes, without deeper knowledge of the Unreal Engine with relative ease. Many of the provided functions are based on existing Unreal Engine tools, but are simplified or wrapped to make them easier to use.

Plugin expects three input files:

- Height map - a square 16-bit gray scale image, where the darkest pixel corresponds to the lowest elevation and the brightest pixel corresponds the highest elevation.
- Segmentation - a square RGB image in which each color represents different biome or region.
- Segmentation legend - a JSON file in which each entry contains three parameters: region name, region color, and region type. The currently implemented region types are BIOME, RIVER, LAKE, and ROAD.

The distinction between these region types is necessary because each type corresponds to a different generation workflow and a different type of Unreal Engine object.

The user can then create the landscape, landscape splines representing roads, river and lake splines, as well as biome splines with basic control over PCG Biome Core. The plugin provides a fast and accessible workflow for environment creation, resulting in a substantial reduction in the time required to prepare a level. After generation, the user can further refine the result using built-in Unreal Engine tools.

The plugin is organized into several pages, each corresponding to a specific workflow within Unreal Engine. These pages are:

- **Startup page** - loading of all input files.
- **Landscape page** - creation of a landscape from a height map.

- **Road spline page** - creation of landscape splines for roads.
- **Water body page** - creation of river and lake splines.
- **Biome page** - creation of PCG Biome Core objects as well as biome splines.

The Road Spline, Water Body, and Biome pages share a common part of the processing pipeline, namely segmentation processing and spline calculation.

3.2 Landscape Creation

This plugin simplifies the process of landscape creation. Its functionality is similar to the built-in Unreal Engine landscape creation workflow. It also includes height map upscaling, but it requires a square-shaped height map. During upscaling, the user can choose the target resolution from a predefined set of valid values. The resolutions used for landscape generation in Unreal Engine 5 must satisfy the following requirements:

$$\begin{aligned} W - 1 &\equiv 0 \pmod{S_Q} \quad , \\ H - 1 &\equiv 0 \pmod{S_Q} \quad , \end{aligned} \tag{3.1}$$

where quad size of component $S_Q = N_S \times S_S$, S_S is quad size of one subsection, and N_S is number of subsections. Width and height of the height map are represented as W and H .

Upscaling of the landscape height map is performed using bilinear interpolation. For each pixel of the destination height map, the corresponding continuous position in the source height map is computed. Since this position usually lies between four neighboring source pixels, the resulting height value is obtained by interpolating between these four samples according to their relative distances.

The scaling factors are chosen so that the boundary pixels of the source and destination heightmaps remain aligned. After interpolation, the computed value is rounded, clamped to the valid 16-bit range, and stored in the destination heightmap. This produces a smoother result than nearest-neighbor upscaling, although it does not introduce any new terrain detail.

A landscape generated directly from a height map is limited by the minimum and maximum values present in that height map. Consequently, the terrain cannot be deformed above or below this range without additional preprocessing. This becomes problematic when generating rivers and lakes, since these features deform the landscape surface.

This is achieved by rescaling all height values so that the terrain includes a user-defined extension above the highest point and below the lowest point. After this adjustment, the scale in the Z axis is recalculated so that the resulting landscape retains the correct real-world size.

The remaining settings are equivalent to those of the built-in landscape creation tool, including position, rotation, scale, and the material assigned to the generated landscape. Figure 3.1 shows the landscape page of the developed plugin.

3.3 Segmentation Processing

The initial step in the creation of all spline-based Unreal Engine objects is segmentation processing. In this step, the plugin applies the algorithms introduced in Chapter 2 to process the segmentation image. This includes segmentation-based binary mask extraction, optional binary mask smoothing, polyline simplification using the Douglas–Peucker algorithm, and

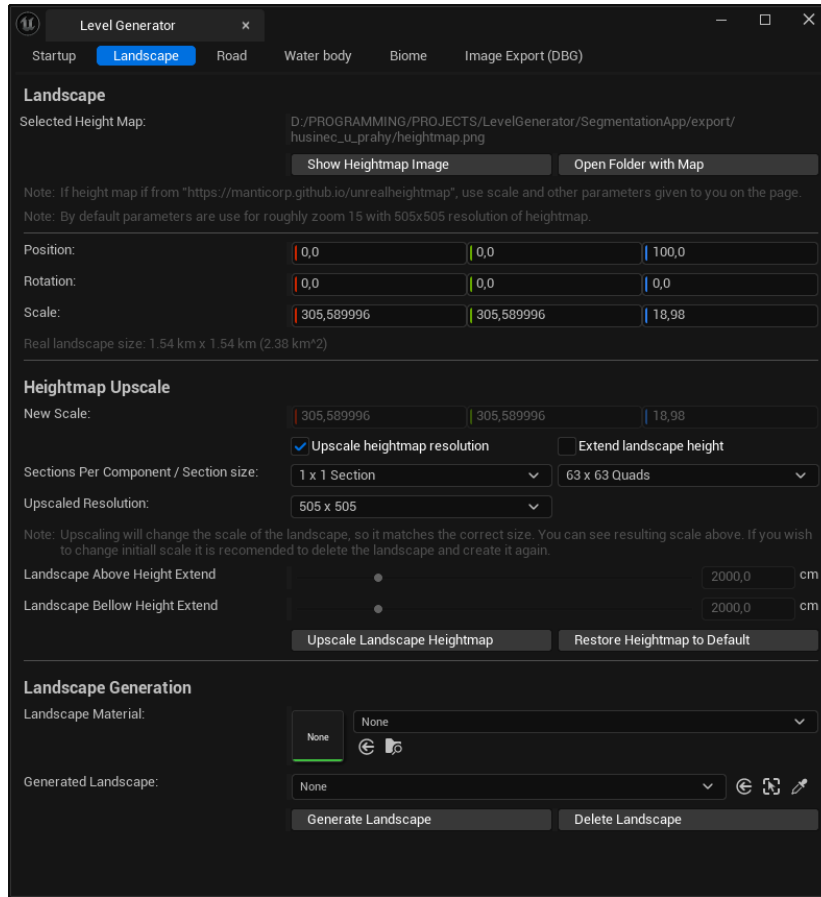
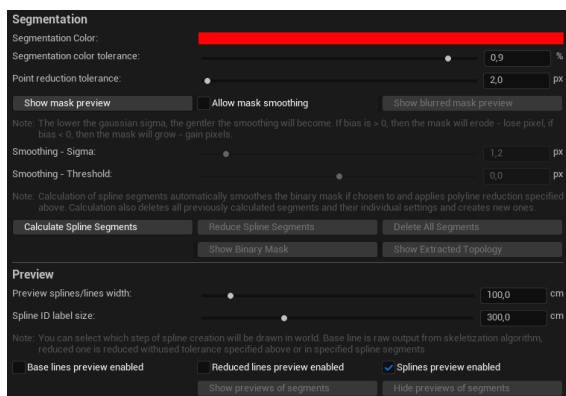
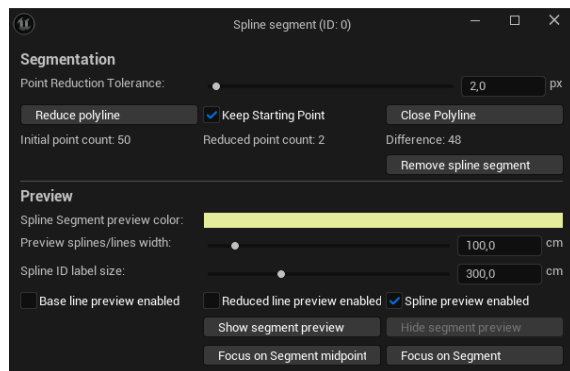


Figure 3.1: Landscape page of the plugin user interface. The page allows user to upscale landscape height map, set landscape properties and material, and landscape creation.



(a) Segmentation page part of all splines.



(b) Segmentation page part of individual spline segment.

Figure 3.2: Figure (a) shows plugin user interface page part of spline creation from segmentation image. Figure (b) shows plugin user interface page part of individual spline segment adjustment based on already generated splines in (a).

subsequent spline interpolation. The plugin also provides spline preview functionality, allowing the user to inspect the generated splines and adjust the parameters accordingly.

The 2D spline, calculated either from a binary mask skeleton or from a contour, is then transformed from binary mask space to landscape space. This process is performed using the Hermite spline representation rather than the Bézier representation, since Hermite splines are more convenient for this type of transformation, as seen on Fig. 3.3.

To transform a point from image space to landscape space, the x and y coordinates are first mapped according to Algorithm 10.

Algorithm 3: Point transformation from image space to landscape space

```

1: procedure TRANSFORM_POINT(image_point, image, landscape)
2:    $min_x, min_y, max_x, max_y = landscape.get\_extends()$ 
3:
4:    $u = image\_point.x / (image.width - 1)$ 
5:    $v = image\_point.y / (image.height - 1)$ 
6:
7:    $q_x = min_x + (max_x - min_x) \cdot u$ 
8:    $q_y = min_y + (max_y - min_y) \cdot v$ 
9:   return ( $q_x, q_y, 0$ )
10: end procedure

```

After this planar transformation, a ray parallel to the z -axis is cast from the transformed point. The intersection of this ray with the landscape determines the final spline point in landscape space. In Unreal Engine 5, this operation also provides the landscape normal vector at the point of intersection.

The landscape normal vector $\mathbf{n}_i$ is then used to project the Hermite tangent vector $\mathbf{M}_i$ onto the tangent plane of the landscape surface. For each spline point, the corrected tangent is computed as

$$\mathbf{M}'_i = \mathbf{M}_i - (\mathbf{M}_i \cdot \mathbf{n}_i)\mathbf{n}_i, \quad (3.2)$$

where $i \in \{0, 1\}$ for a single Hermite curve segment.

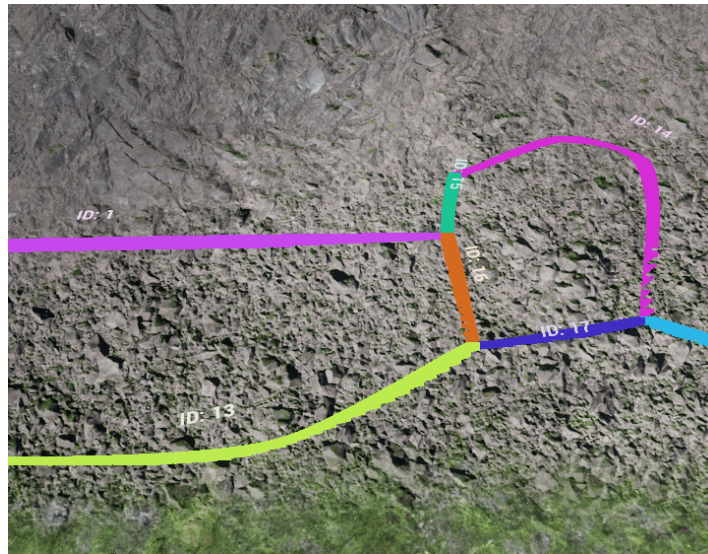


Figure 3.3: Example of spline preview functionality.

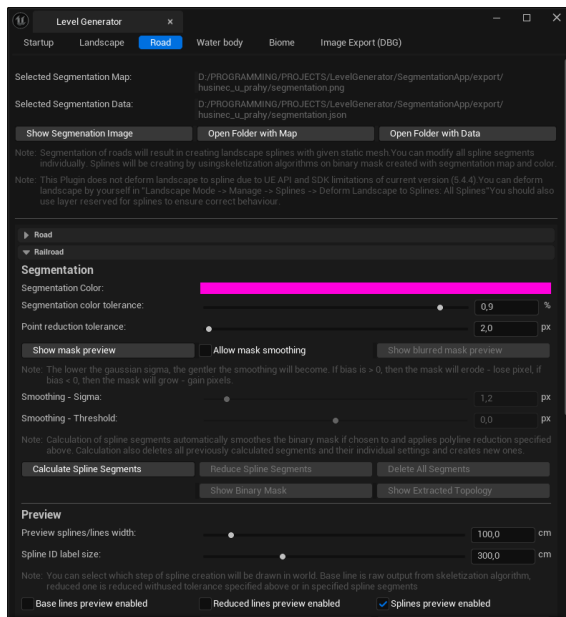
Figure 3.2a shows the segmentation-processing part shared by all plugin pages that generate splines. In addition to global processing parameters, the plugin also allows individual adjustment of already generated spline segments, as shown in Fig. 3.2b.

3.4 Unreal Engine Spline Creation Pages

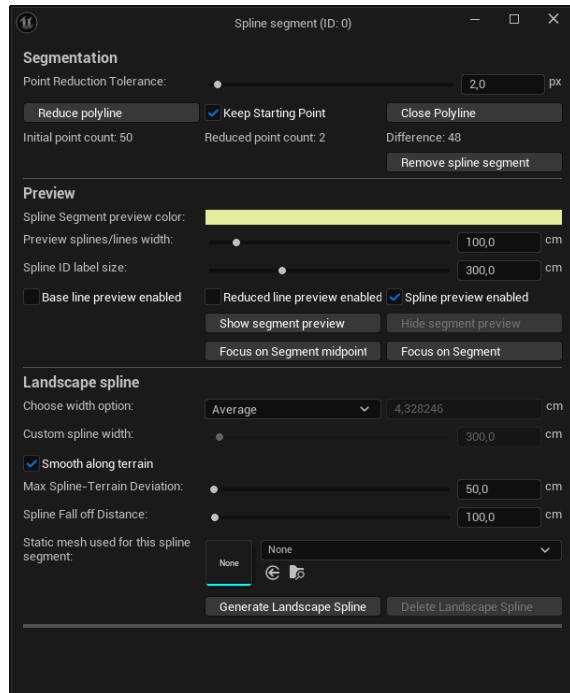
3.4.1 Landscape Spline Creation

The purpose of this page is to create landscape splines on a previously generated landscape. All segmentation regions of type ROAD are displayed in the user interface as a list of collapsible sections, as shown in Fig. 3.4a. Each section contains a segmentation processing part and a landscape spline creation part. User can adjust individual generated splines as shown in Fig. 3.4b.

The segmentation processing part was described in detail in Section 3.3. In addition, the page contains a landscape spline configuration part. In this section, the user can specify the width of the landscape spline and its falloff distance. The width can be determined automatically from the EDT of the binary mask skeleton or specified manually by the user. The falloff distance defines the distance from the spline at which landscape deformation begins.



(a) Landscape spline page with two segmentation regions of type ROAD: Road and Railroad.



(b) Landscape spline page for individual spline

Figure 3.4: Figure (a) shows plugin user interface page part of a landscape spline creation from segmentation image. Figure (b) shows plugin user interface page part of individual spline segment adjustment based on already generated splines in (a).

The user can enable an option to smooth the landscape spline with respect to the terrain surface. This operation uses de Casteljau's algorithm, described in Section 2.5.3. The

smoothing algorithm evaluates the midpoint of a spline segment and casts a ray parallel to the z -axis in order to find its intersection with the landscape. If the distance between the original midpoint and the intersection point exceeds a user-defined threshold, the spline segment is split into two segments and the midpoint is projected onto the landscape as described in Section 3.3. This process is repeated until the distance for every segment is below the specified threshold. Result of such smoothing can be seen on Fig. 3.5

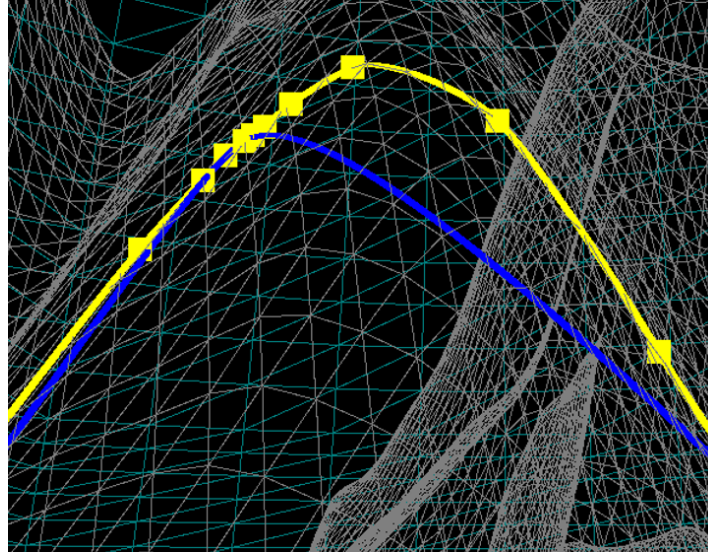


Figure 3.5: Example of spline smoothing with respect to the terrain functionality. Blue spline is the initial and unsmoothed. Yellow spline is smoothed with respect to the terrain. Yellow rectangles are added points of the spline to achieve this result.

In Unreal Engine 5, version 5.5 and later, automatic deformation of the landscape according to the landscape spline is supported through the C++ Application Programming Interface (API). However, this project was developed in Unreal Engine 5 version 5.4.4, where this functionality was not available. For this reason, the plugin provides the user with detailed instructions for performing this step manually.

Result of landscape spline generation with landscape deformation is seen on Fig. 3.6. It can be observed that small manual adjustments are needed.

3.4.2 River and Lake Spline Creation

The purpose of this page is to create river splines or lake splines on a previously generated landscape. Landscape height extension is recommended, as the depth of rivers and lakes can extend below the lowest point of the landscape. Similarly to section 3.4.1, all segmentation regions of types RIVER or LAKE are displayed in the user interface as a list of collapsible sections, shown in Fig. 3.7a. Each section contains a segmentation processing part and a water body part. User can adjust individual generated splines as shown in Fig. 3.7b.

Water zone must be created before user can create water body splines. Water zone encapsulates the entire landscape and its vertical extends. This object is needed for correct creation of water body splines and its usage. Before calculating splines it is recommended to perform binary mask smoothing, otherwise many spline branches can be generated, as seen in Fig. 2.5a. If the binary mask skeleton or outline is generated fragmented even after the binary

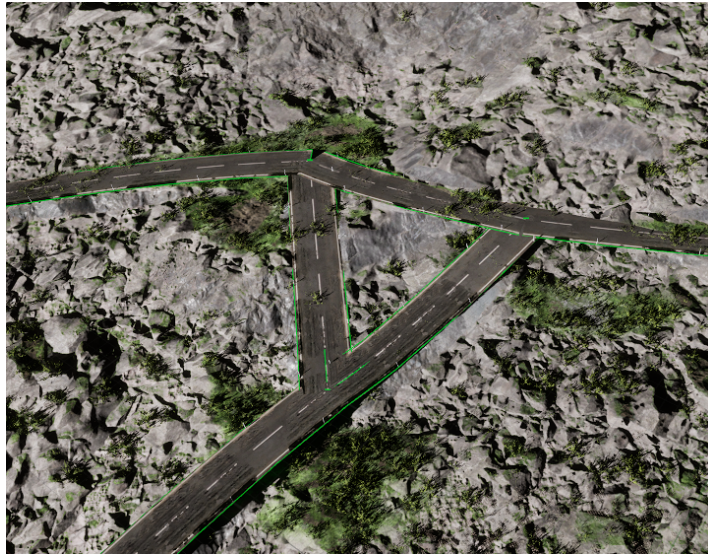


Figure 3.6: Example of generated landscape spline from binary mask skeleton.

mask is smoothed out, user can choose to stitch many spline segments together to form one continuous spline or to close opened splines.

The

3.4.3 PCG Biome Core Creation

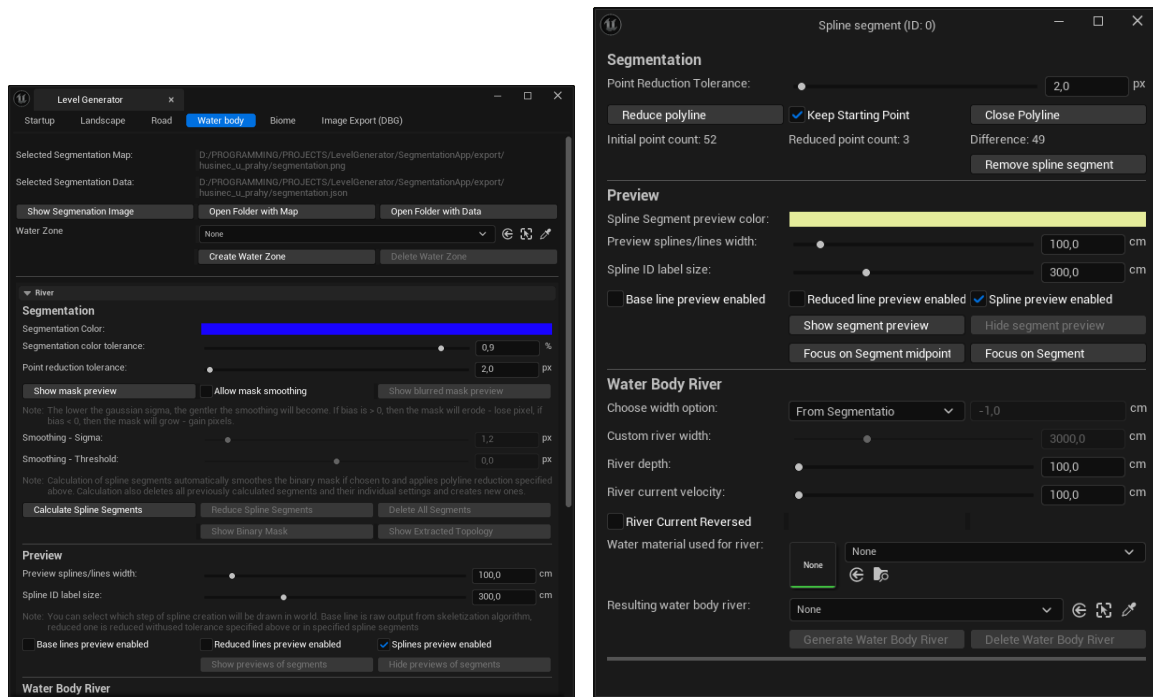
The Biome page follows the workflow of the PCG Biome Core system [7]. The user first needs to create a PCG Biome Core object. The plugin user interface exposes only the most relevant settings, such as the PCG Graph, which controls how the procedural generation is performed, the biome blending range, and the cache cell size. Increasing the cache cell size may help reduce the performance overhead of the PCG Biome Core system.

The plugin user interface also provides controls for managing the PCG Biome Core object. The user can manually generate, update, or refresh biomes, as well as clean up previously generated biome instances. When working with large and complex biomes, it is recommended to disable automatic refresh of the PCG Biome Core object, since frequent updates may overload the Unreal Engine editor and lead to instability. The available settings are shown in Fig. 3.9a.

The PCG Biome Core system is computationally demanding, especially in terms of memory usage. During testing, large biome setups occasionally led to instability of the Unreal Engine 5 editor. Several mitigation strategies were tested, but none of them significantly improved the observed behavior.

3.4.4 Biome Spline Creation

Biome spline generation requires a Biome Definition and one or more Biome Assets, as shown in Fig. 3.9b. Both are based on the Data Asset type in Unreal Engine 5 and are provided by the PCG Biome Core plugin. The Biome Definition specifies the name and color of the biome. This step is required by all biome construction methods available in the PCG



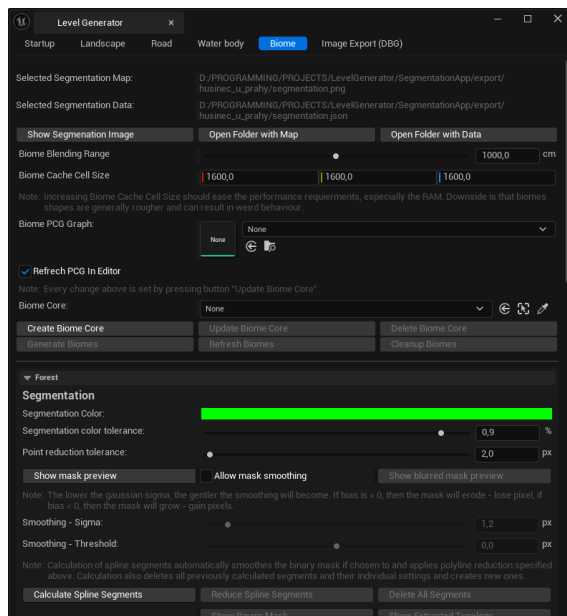
(a) Water body page with region of type RIVER: River.

(b) Water body page for individual spline

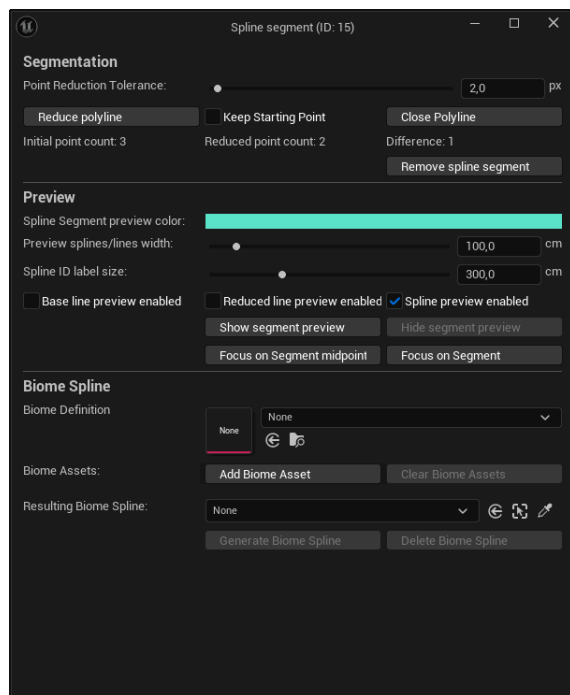
Figure 3.7: Figure (a) shows plugin user interface page part of a water body spline creation from segmentation image. Figure (b) shows plugin user interface page part of individual spline segment adjustment based on already generated splines in (a).



Figure 3.8: Example of generated river spline from binary mask skeleton. At each point of the spline different width, calculated using EDT, was chosen giving more accurate and realistic look of the river.



(a) Biome page with two segmentation regions of type BIOME: Forest and Quarry.

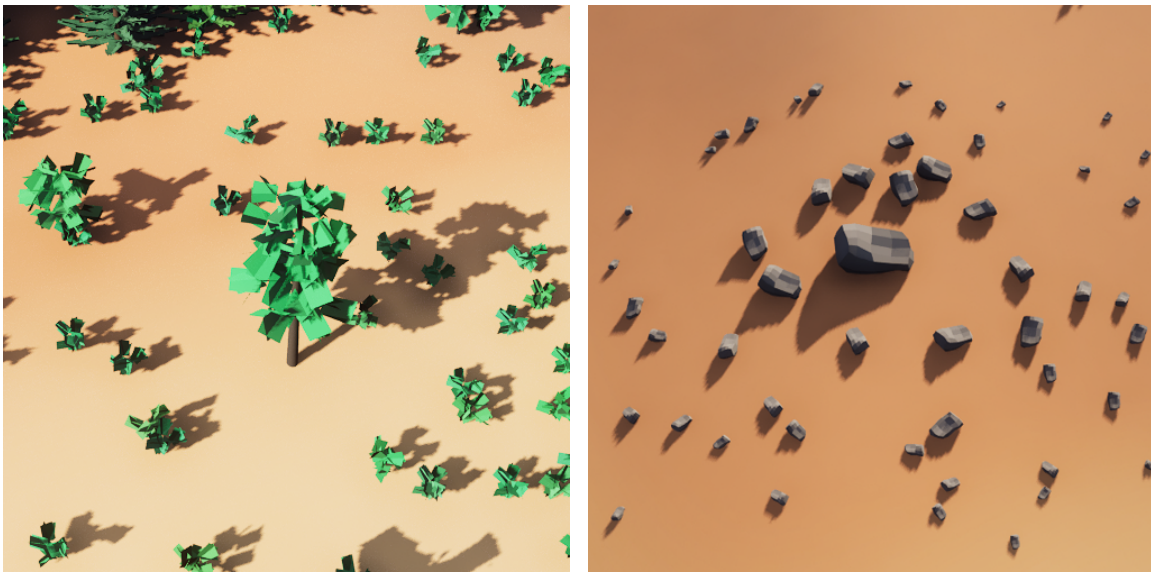


(b) Biome page for individual spline

Figure 3.9: Figure (a) shows plugin user interface page part of a biome spline creation from segmentation image. Figure (b) shows plugin user interface page part of individual spline segment adjustment based on already generated splines in (a).

Biome Core plugin, and it is especially important when biomes are generated from a biome mask. For biome spline generation alone, however, the biome name and color are not used directly. Generated biome using biome spline can be seen on Fig. 3.11.

A Biome Asset defines how the biome should be generated. This Data Asset contains settings for the static meshes used in the biome, such as trees and rocks, together with their configuration, probability weights, collision settings, the generator represented by a PCG Graph, optional child biome assets, and other parameters. Child Biome Assets work similarly to their parent assets, but only within a limited distance from the parent asset instances. This makes it possible to create more realistic groupings of generated objects, such as a tree surrounded by saplings, as on Fig. 3.10a or a large rock surrounded by smaller stones, as on Fig. 3.10b.



(a) Tree surrounded by saplings generated through PCG and Biome Asset. (b) Rock surrounded by stones generated through PCG and Biome Asset

Figure 3.10: Example of PCG generation through Biome Asset of tree surrounded by saplings (a) and rock surrounded by stones (b).

The generating PCG Graph directly affects how biome objects are distributed. This functionality is provided by the PCG Framework plugin [8]. A typical PCG Graph first generates a large number of random points inside an enclosed area, which in this case is defined by the biome spline. These points are assigned random rotations and scales. The graph then processes the points through a sequence of filters defined either in the PCG Graph itself or in the Biome Asset. These filters modify the number of points or their properties. In the final step, static meshes are spawned at the resulting point locations. This allows the user to prepare different mesh placement procedures depending on the intended biome appearance.

3.5 Generated Worlds

As a proof of concept of this plugin, four different worlds were generated. Environments were chosen to be distinct to showcase what possibilities can this plugin accomplish. All

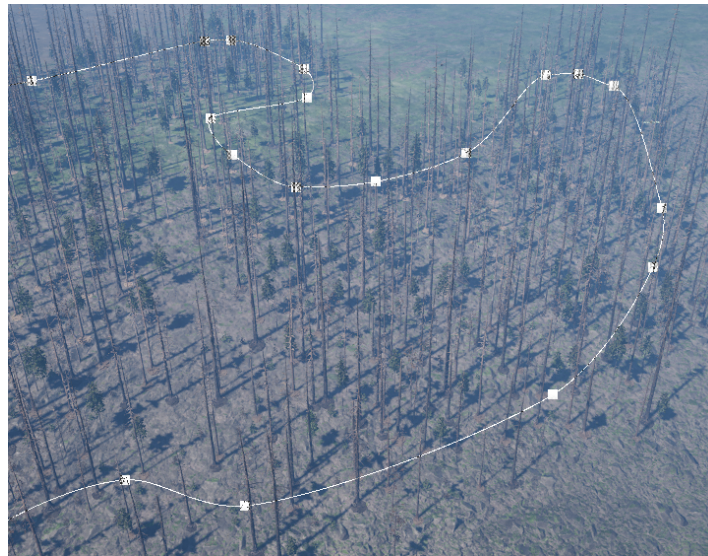


Figure 3.11: Example of generated forest using biome spline in Unreal Engine 5 plugin PCG Biome Core.

four worlds hat their height map generated from Unreal Engine 5 height map generator and segmentation was done manually. Following four worlds were generated:

- Desert Oasis - shown on Fig. 3.12.
- Étretat Cliffs - cliffs in northern France shown on Fig. 3.13.
- Grand Canyon - canyon in America shown on Fig. 3.14.
- Stuðlagil Canyon - canyon in Iceland shown on Fig. 3.15.

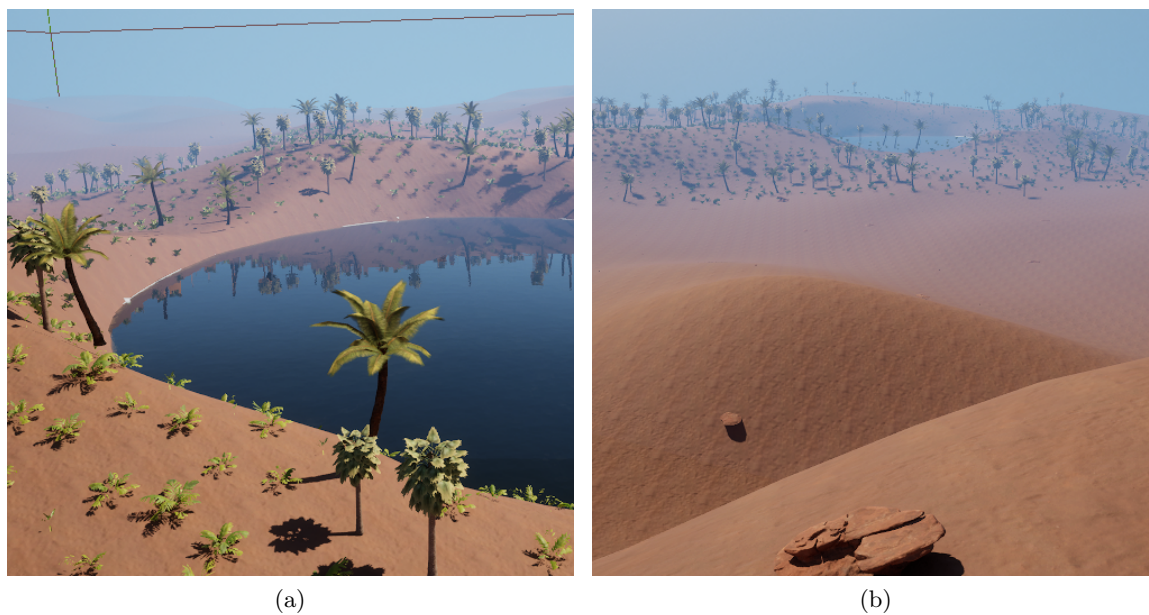


Figure 3.12: Example world named Desert Oasis generated using by this Unreal Engine 5 plugin.

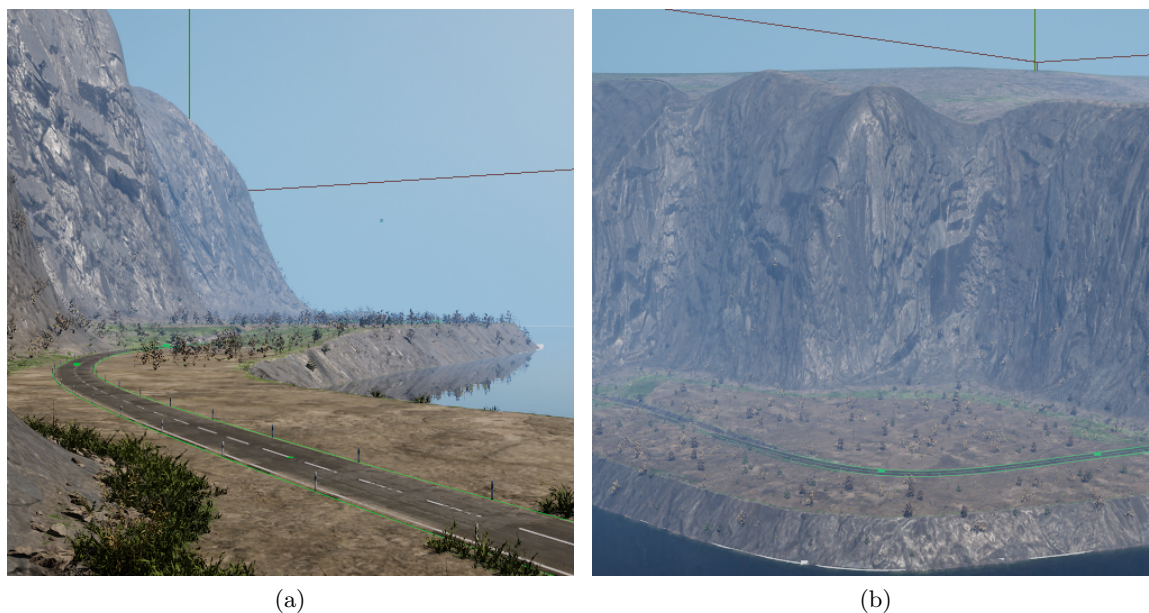


Figure 3.13: Example world named Étretat Cliffs generated using by this Unreal Engine 5 plugin.



Figure 3.14: Example world named Grand Canyon generated using by this Unreal Engine 5 plugin.

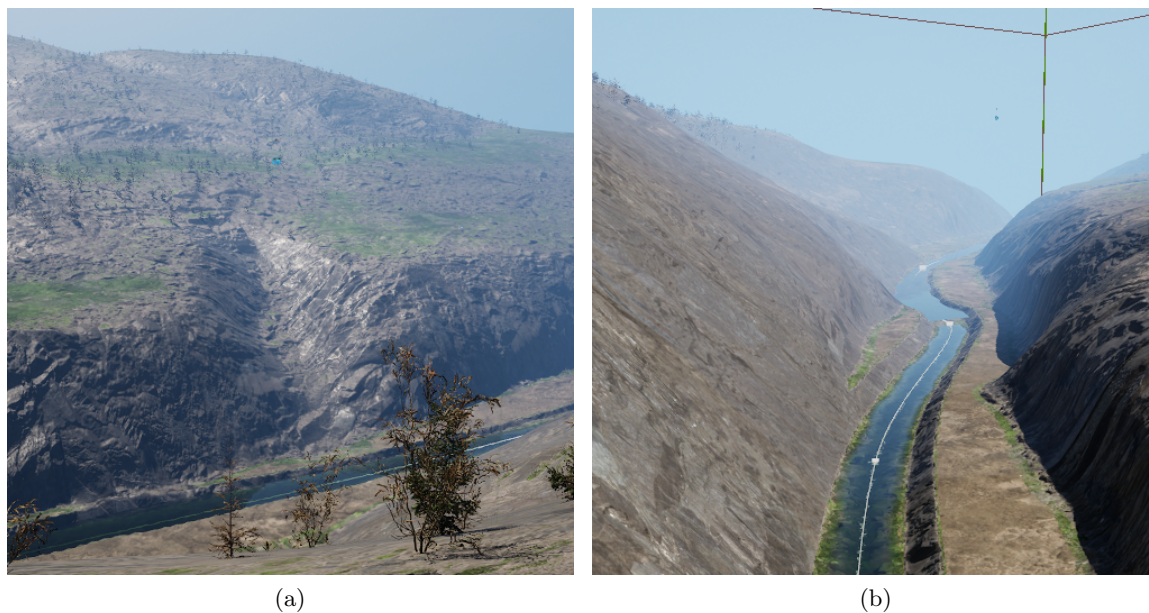


Figure 3.15: Example world named Stuðlagil Canyon generated using by this Unreal Engine 5 plugin.

Chapter 4

Conclusion

Summarize the achieved results. Can be similar as an abstract or an introduction, however, it should be written in past tense.

Chapter 5

References

- [1] D. Čapek et al., *FlightForge: Advancing UAV Research with Procedural Generation of High-Fidelity Simulation and Integrated Autonomy*, Publication Title: arXiv preprint arXiv: 2502.05038, 2025. [Online]. Available: <https://arxiv.org/abs/2502.05038v1>
- [2] H. Jain and A. P. Kumar, *A Sequential Thinning Algorithm For Multi-Dimensional Binary Patterns*, en, Oct. 2017. Accessed: Mar. 29, 2026. [Online]. Available: <https://arxiv.org/abs/1710.03025v2>
- [3] P. F. Felzenszwalb and D. P. Huttenlocher, “Distance Transforms of Sampled Functions,” *Theory of Computing*, vol. 8, pp. 415–428, Sep. 2012, Number: 19. DOI: [10.4086/toc.2012.v008a019](https://doi.org/10.4086/toc.2012.v008a019) Accessed: Mar. 29, 2026. [Online]. Available: <https://theoryofcomputing.org/articles/v008a019/>
- [4] Z. Guo and R. W. Hall, “Fast fully parallel thinning algorithms,” *CVGIP: Image Understanding*, vol. 55, no. 3, pp. 317–328, May 1992, ISSN: 1049-9660. DOI: [10.1016/1049-9660\(92\)90029-3](https://doi.org/10.1016/1049-9660(92)90029-3) Accessed: Mar. 29, 2026. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/1049966092900293>
- [5] E. T. Y. Lee, “Choosing nodes in parametric curve interpolation,” *Computer-Aided Design*, vol. 21, no. 6, pp. 363–370, Jul. 1989, ISSN: 0010-4485. DOI: [10.1016/0010-4485\(89\)90003-1](https://doi.org/10.1016/0010-4485(89)90003-1) Accessed: Apr. 9, 2026. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/0010448589900031>
- [6] D. H. Douglas and T. K. Peucker, *ALGORITHMS FOR THE REDUCTION OF THE NUMBER OF POINTS REQUIRED TO REPRESENT A DIGITIZED LINE OR ITS CARICATURE / Cartographica*. Accessed: Mar. 29, 2026. [Online]. Available: <https://utppublishing.com/doi/abs/10.3138/fm57-6770-u75u-7727>
- [7] Epic Games, *Procedural Content Generation Biome Core Overview*, en-US. Accessed: May 2, 2026. [Online]. Available: <https://dev.epicgames.com/documentation/unreal-engine/procedural-content-generation-pcg-biome-core-and-sample-plugins-overview-guide-in-unreal-engine>
- [8] Epic Games, *Procedural Content Generation Overview*, en-US. Accessed: May 2, 2026. [Online]. Available: <https://dev.epicgames.com/documentation/unreal-engine/procedural-content-generation-overview>
- [9] C. Yuksel, S. Schaefer, and J. Keyser, *On the parameterization of Catmull-Rom curves / 2009 SIAM/ACM Joint Conference on Geometric and Physical Modeling*. Accessed: Mar. 29, 2026. [Online]. Available: <https://dl.acm.org/doi/10.1145/1629255.1629262>
- [10] T. Y. Zhang and C. Y. Suen, *A fast parallel algorithm for thinning digital patterns / Communications of the ACM*. Accessed: Mar. 29, 2026. [Online]. Available: <https://dl.acm.org/doi/10.1145/357994.358023>